Learn Python and Machine Learning by Building an FPL Squad Picker

A 70-day, one-hour-a-day course for complete beginners

Welcome

You do not need any technical background to use this book. You only need a computer, an internet connection, and one hour a day for ten weeks. Every lesson teaches one small idea and then makes you use it on real Fantasy Premier League (FPL) data, so by Day 70 you will have a working web app you can show to anyone.

What you will build: the FPL Smart Squad Picker. A small website with four pages:

1. **Player Explorer**: search and filter every FPL player.
2. **Value Finder**: shows which players give the most points per £1m.
3. **Fixture Heatmap**: shows which teams have easy or hard upcoming matches.
4. **Squad Builder**: predicts player points with machine learning, then picks the best legal 15-player squad within the £100m budget.

What you will be able to do afterwards: write Python, pull data from an API, analyse it with pandas, draw charts, train a prediction model, solve an optimization problem, and publish a project on GitHub with a live link.

How this book works

Every lesson is built to fit in **60 minutes**:

Block	Time	What you do
Read	15 min	Read the idea. Do not memorize anything.
Type	30 min	Type the code yourself. Copy-paste teaches far less than typing.
Check	10 min	Complete the checkpoint. If you cannot, repeat the Type block.
Note	5 min	Write one line in your learning journal: what I learned, what confused me.

Five rules

1. **Type the code, do not paste it.** Typos are part of learning.
2. **Errors are normal.** Read the last line of the error message. If you are stuck for more than 15 minutes, write the error in your journal and move on. Come back the next day.
3. **Day 7 of each week is a Project Day.** You combine the week's lessons into one piece of the final project.
4. **Never skip a checkpoint.** It is how you know the lesson worked.
5. **Do not fall behind by more than two days.** If you miss days, repeat the last lesson and continue. Do not try to catch up with a 3-hour session.

The numbers in the small examples (such as prices and points) are made up for practice. Real FPL data appears from Week 2 onwards.

Tools you will use (all free)

Tool	What it is	When you need it
Google account	Lets you use Colab and Drive	Day 1
Google Colab	A programming notebook that runs in your browser, with nothing to install	Weeks 1-8
GitHub account	A website where you store and show your code	Week 9
VS Code	A free program for writing code on your own computer	Week 9

Tool	What it is	When you need it
Streamlit	A Python tool that turns your code into a website	Weeks 5-10

Course map

Part	Weeks	Days	You learn	Project piece
1. Python basics	1	1-7	Variables, lists, dictionaries, loops, functions	Mini player cards
2. FPL data and pandas	2-3	8-21	APIs, tables, filtering, merging, new columns	Clean player table and value ranking
3. Charts and dashboard	4-5	22-35	Charts, fixtures, Streamlit	First web dashboard
4. Machine learning	6-7	36-49	Features, models, honest testing	Points predictor
5. Optimization	8	50-56	Rules and constraints, linear programming	Squad optimizer
6. App and publishing	9-10	57-70	Combining, Git, GitHub, deployment	Live Smart Squad Picker

Part 1: Python Basics

Week 1: Your First Programs

Day 1: Set up and say hello

Goal: run your first line of code.

Read. A *program* is a list of instructions the computer follows one line at a time. *Python* is the language we will use. *Google Colab* lets you write and run Python in a web page, so you do not install anything.

Type.

1. Go to colab.research.google.com and sign in with your Google account.
2. Click **New notebook**. Rename it `fpl-day01`.
3. In the first box (called a *cell*) type the following, then press **Shift + Enter** to run it:

```
print('Hello FPL')
name = 'Salah'
print(name)
```

Check. Change `Salah` to another player and run again. The new name should print.

Note. What does `print` do, in your own words?

Day 2: Variables and data types

Goal: store information in named boxes.

Read. A *variable* is a labelled box that holds a value. Python has three kinds you will use constantly: **text** (`'Haaland'`), **whole numbers** (`20`), and **decimal numbers** (`14.5`). There is also **True/False**.

Type.

```
player = 'Haaland'
price = 14.5
total_points = 150
is_injured = False

value = total_points / price
print(player, 'costs', price, 'million')
print('Points per million:', round(value, 2))
print(f'{player} gives {value:.1f} points per million')
```

The `f'...'` form is called an *f-string*. It lets you put variables inside text using curly brackets.

Check. Make a second player with different numbers and print a sentence about them.

Note. Which line calculated the value, and how?

Day 3: Lists

Goal: hold many values together.

Read. A *list* is an ordered collection written in square brackets. The first item is number **0**, not 1. This surprises everyone once.

Type.

```
prices = [14.5, 12.5, 9.0, 5.5]

print(prices[0])        # first item
print(prices[-1])       # last item
print(len(prices))      # how many items
print(max(prices), min(prices))

prices.append(7.0)      # add an item
print(sorted(prices))   # sorted copy
print(sum(prices) / len(prices))  # average
```

Anything after `#` is a *comment*. Python ignores it, and it is there for humans.

Check. Print the second-cheapest price using `sorted` and a position number.

Note. Why is the first item `[0]`?

Day 4: Dictionaries

Goal: describe one thing with several facts.

Read. A *dictionary* stores facts under labels (called *keys*), written in curly brackets. The FPL API sends data in exactly this shape, so this lesson matters.

Type.

```
player = {
    'name': 'Saka',
    'team': 'Arsenal',
    'price': 10.0,
    'points': 120
}

print(player['name'])
player['goals'] = 9          # add a new fact
player['price'] = 10.1       # change a fact
print(player)

players = [
    {'name': 'Saka', 'price': 10.0, 'points': 120},
    {'name': 'Palmer', 'price': 10.5, 'points': 135},
    {'name': 'Watkins', 'price': 9.0, 'points': 98},
]
print(players[1]['name'])
```

Check. Add a fourth player to the `players` list and print their points.

Note. What is the difference between a list and a dictionary?

Day 5: Decisions and loops

Goal: make the computer repeat work and make choices.

Read. A *loop* repeats instructions for every item. An *if* statement runs code only when something is true. Python uses **indentation** (4 spaces) to show which lines belong inside.

Type.

```
players = [
    {'name': 'Saka', 'price': 10.0, 'points': 120},
    {'name': 'Palmer', 'price': 10.5, 'points': 135},
    {'name': 'Watkins', 'price': 9.0, 'points': 98},
    {'name': 'Mbeumo', 'price': 7.5, 'points': 110},
]

for p in players:
    if p['price'] >= 10:
        label = 'premium'
    else:
        label = 'budget'
    print(p['name'], label)
```

Check. Change the rule so players with 115 or more points print `star`, and everyone else prints `regular`.

Note. What does the indentation do?

Day 6: Functions

Goal: package a calculation so you can reuse it.

Read. A *function* is a named recipe. You give it inputs, it gives back an output with `return`. You write the recipe once and use it many times.

Type.

```
def points_per_million(points, price):
    return round(points / price, 2)

print(points_per_million(120, 10.0))
print(points_per_million(110, 7.5))

for p in players:
    print(p['name'], points_per_million(p['points'], p['price']))
```

Check. Write a function `is_bargain(price)` that returns `True` when the price is under 8.0, and test it on two prices.

Note. Why is a function better than copying the same formula five times?

Day 7: Project Day. Mini player cards

Goal: combine Week 1 into one small program.

Task. Using a list of at least 6 players (dictionaries with name, team, price, points):

1. Write a function that returns points per million.
2. Loop through the players and print a card for each:

```
Palmer (Chelsea) | 10.5m | 135 pts | value 12.86
```

3. After the loop, print the name of the best-value player.

Hints. Keep a variable called `best_player` and update it inside the loop with an `if`. Build the card line with an f-string.

Done when: the program runs from top to bottom without errors and names a best-value player.

Save your work. In Colab choose **File, then Save a copy in Drive**.

Week 2: Talking to the FPL API

Day 8: What is an API?

Goal: download real FPL data.

Read. An *API* is a web address that returns data instead of a web page. The FPL website has one that anyone can read. When you ask for data, the server replies with a *status code*: **200** means success.

Type.

```
import requests

url = 'https://fantasy.premierleague.com/api/bootstrap-static/'
response = requests.get(url, timeout=10)

print(response.status_code)
data = response.json()
print(type(data))
print(data.keys())
```

`import requests` loads a ready-made tool for downloading from the web.

Check. You should see `200` and a list of keys including `elements`, `teams`, and `events`.

Note. What does status code 200 mean?

Day 9: Exploring the data

Goal: understand what you downloaded.

Read. The data is a big dictionary. `elements` is a list of every player, `teams` is a list of the 20 clubs, and `events` is a list of gameweeks. Prices come as **tenths of a million**, so `100` means £10.0m.

Type.

```
print(len(data['elements']))      # number of players
first = data['elements'][0]
print(first.keys())
print(first['web_name'], first['now_cost'], first['total_points'])
```

Fields you will use often:

Field	Meaning
`web_name`	Player's display name
`team`	Team number (match it with `teams`)
`element_type`	Position: 1 GK, 2 DEF, 3 MID, 4 FWD
`now_cost`	Price in tenths of £m
`total_points`	Points this season
`minutes`	Minutes played
`form`	Recent average points
`selected_by_percent`	Ownership
`goals_scored`, `assists`	Attacking returns

Check. Print the number of teams and the name of the first team in `data['teams']`.

Day 10: Looping over real players

Goal: use Week 1 skills on real data.

Type.

```
for p in data['elements'][:10]:
    print(p['web_name'], p['now_cost'] / 10, p['total_points'])

# find a player by name
for p in data['elements']:
    if p['web_name'] == 'Salah':
        print(p['web_name'], p['total_points'], p['form'])
```

Check. Find a different player and print their price in millions.

Note. Why do we divide `now_cost` by 10?

Day 11: Your first table with pandas

Goal: turn a messy list into a table.

Read. *pandas* is the standard Python tool for tables. A table is called a **DataFrame**. Think of it as a spreadsheet that you control with code.

Type.

```
import pandas as pd

players = pd.DataFrame(data['elements'])
print(players.shape)        # (rows, columns)
players.head()              # first 5 rows
print(list(players.columns))
```

Check. How many rows and columns does the table have? Write the numbers in your journal.

Day 12: Choosing columns and making new ones

Goal: keep only what you need and add calculated columns.

Type.

```
df = players[['web_name', 'team', 'element_type', 'now_cost',
              'total_points', 'minutes', 'form']].copy()

df['price'] = df['now_cost'] / 10
df['points_per_million'] = df['total_points'] / df['price']
df.head(10)
```

Check. Add a column `points_per_90` calculated as `total_points / minutes * 90`. Some rows may show strange values for players with 0 minutes. Note this in your journal, because we will deal with it on Day 15.

Day 13: Sorting and filtering

Goal: ask questions of your table.

Type.

```
# top 10 by points
df.sort_values('total_points', ascending=False).head(10)

# players cheaper than 6.0
df[df['price'] < 6.0]

# midfielders under 8.0 (element_type 3 means midfielder)
cheap_mids = df[(df['element_type'] == 3) & (df['price'] < 8.0)]
cheap_mids.sort_values('total_points', ascending=False).head(10)
```

Use `&` for **and**, `|` for **or**, and wrap each condition in brackets.

Check. Find the five best-scoring defenders under 5.5.

Day 14: Project Day. Clean player table

Goal: build the first real piece of the project.

Task.

1. Download the data and build the `players` DataFrame.
2. Build a `teams` table and attach team names:

```
teams = pd.DataFrame(data['teams'])[['id', 'name']]
teams = teams.rename(columns={'id': 'team', 'name': 'team_name'})
df = df.merge(teams, on='team')
```

3. Add a readable position column:

```
df['position'] = df['element_type'].map({1: 'GK', 2: 'DEF', 3: 'MID', 4:
'FWD'})
```

4. Print the top 10 players overall and the top 5 in each position.
5. Save the table: `df.to_csv('fpl_players.csv', index=False)`, then download it from the Colab file panel (left side, folder icon). Colab deletes files when your session ends, so download it.

Done when: you have a CSV with player name, team name, position, price, points, and points per million.

Milestone. You have finished Part 1 and the first half of Part 2. You can already answer real FPL questions with code.

Week 3: Cleaning, Grouping and Fixtures

Day 15: Fixing messy data

Goal: make the table safe to calculate with.

Read. Real data is messy. Two problems you already met: some numbers arrive as *text* (like `'5.4'`), and some players have 0 minutes, which breaks divisions. Cleaning means fixing these before analysis.

Type. Reuse your Day 14 code (or reload `fpl_players.csv`), then:

```
df['form'] = pd.to_numeric(players['form'])
df['selected_by'] = pd.to_numeric(players['selected_by_percent'])

played = df[df['minutes'] > 0].copy()
played['points_per_90'] = played['total_points'] / played['minutes'] * 90
print(len(df), len(played))
played.isna().sum()      # count missing values per column
```

Check. How many players have zero minutes? Write the number in your journal.

Day 16: Grouping

Goal: answer questions like *which team has the most points?*

Read. `groupby` splits the table into groups, calculates something per group, and gives you a summary.

Type.

```
played.groupby('position')['price'].mean().round(2)

team_points = played.groupby('team_name')['total_points'].sum()
team_points.sort_values(ascending=False).head(10)

played.groupby('position').agg(players=('web_name', 'count'),
                               avg_points=('total_points', 'mean'),
                               max_price=('price', 'max')).round(1)
```

Check. Find the average `points_per_90` for each position.

Day 17: The value ranking

Goal: build the Value Finder logic.

Read. Points per million is unfair to players who played 10 minutes. A common fix is a *minimum minutes* rule. You choose the rule, and you should write it down, because it affects the answer.

Type.

```
MIN_MINUTES = 450
value = played[played['minutes'] >= MIN_MINUTES]

for pos in ['GK', 'DEF', 'MID', 'FWD']:
    print('---', pos)
    best = value[value['position'] == pos].sort_values('points_per_million',
ascending=False)
    print(best[['web_name', 'team_name', 'price', 'total_points',
'points_per_million']].head(5).to_string(index=False))
```

Check. Change `MIN_MINUTES` to 900. Which names disappear? Note one in your journal.

Day 18: Downloading fixtures

Goal: get all matches of the season.

Type.

```
BASE = 'https://fantasy.premierleague.com/api/'
fx = pd.DataFrame(requests.get(BASE + 'fixtures/', timeout=10).json())
print(fx.shape)
fx = fx[['event', 'team_h', 'team_a', 'team_h_difficulty',
'team_a_difficulty', 'kickoff_time', 'finished']]
fx.head()
```

`event` is the gameweek number. `team_h` is the home team and `team_a` the away team. Difficulty runs from 1 (easy) to 5 (hard).

Check. How many fixtures are in the table? How many have `finished` equal to `True`?

Day 19: One row per team per match

Goal: reshape fixtures so each team has its own rows.

Read. Each match currently has one row. To ask *how hard is Arsenal's next 5?* we need one row per team per match, so we stack the home and away views.

Type.

```
fx = fx.dropna(subset=['event']).copy()
fx['event'] = fx['event'].astype(int)

home = fx.rename(columns={'team_h': 'team', 'team_a': 'opponent',
'team_h_difficulty': 'difficulty'})
home = home[['event', 'team', 'opponent', 'difficulty']]
home['venue'] = 'H'

away = fx.rename(columns={'team_a': 'team', 'team_h': 'opponent',
'team_a_difficulty': 'difficulty'})
away = away[['event', 'team', 'opponent', 'difficulty']]
away['venue'] = 'A'

fixtures = pd.concat([home, away], ignore_index=True)
fixtures = fixtures.merge(teams.rename(columns={'team': 'team', 'team_name':
'team_name'}), on='team')
print(fixtures.shape)
```

Check. Filter the table to one team and print its next 5 rows sorted by `event`.

Day 20: Next five gameweeks

Goal: compute fixture difficulty for each team.

Type.

```
next_gw = next(e['id'] for e in data['events'] if e['is_next'])
upcoming = fixtures[(fixtures['event'] >= next_gw) & (fixtures['event'] <
next_gw + 5)]

ease = upcoming.groupby('team_name')
['difficulty'].mean().round(2).sort_values()
print('Easiest runs:'); print(ease.head(5))
print('Hardest runs:'); print(ease.tail(5))
```

If you ever run this when no gameweek is flagged as next (such as after the season ends), replace `next_gw` with a number by hand.

Check. Which team has the easiest next five?

Day 21: Project Day. Value and fixtures tables

Task. Put everything from Weeks 2-3 into one clean notebook with these headings (use text cells): *1. Download*, *2. Clean*, *3. Value ranking*, *4. Fixtures*. Then:

1. Create functions `load_players()` and `load_fixtures()` that return clean tables.
2. Save `value_table.csv` (best 20 by points per million with minutes at least 450) and `fixture_ease.csv`.
3. Add a text cell that explains your minimum-minutes choice in two sentences.

Done when: the notebook runs from top to bottom after choosing **Runtime, then Restart and run all**.

Week 4: Charts

Day 22: Bar chart

Goal: draw your first chart.

Read. Charts make patterns visible. `matplotlib` is the classic Python chart tool.

Type.

```
import matplotlib.pyplot as plt

top = played.sort_values('total_points', ascending=False).head(10)
plt.figure(figsize=(8, 5))
plt.barh(top['web_name'], top['total_points'])
plt.gca().invert_yaxis()
plt.title('Top 10 players by points')
plt.xlabel('Points')
plt.show()
```

Check. Make the same chart for the top 10 by `points_per_million` (use the `value` table).

Day 23: Scatter plot

Goal: see the relationship between price and points.

Type.

```
plt.figure(figsize=(8, 5))
plt.scatter(value['price'], value['total_points'], alpha=0.6)
plt.xlabel('Price (million)')
plt.ylabel('Total points')
plt.title('Price vs points')
plt.show()
```

Check. Describe the pattern in one sentence. Where are the dots far above the crowd? Those are the bargains.

Day 24: Histograms and the zero problem

Goal: see how points are distributed.

Type.

```
plt.hist(played['total_points'], bins=30)
plt.title('Season points per player')
plt.show()
```

Read. Most players score few points and a handful score a lot. This *skewed* shape matters later, because prediction models find such data hard.

Check. Draw one histogram per position using a loop.

Day 25: Interactive charts with Plotly

Goal: make charts you can hover over.

Type.

```
import plotly.express as px

fig = px.scatter(value, x='price', y='total_points', color='position',
                 hover_name='web_name', hover_data=['team_name', 'form'])
fig.update_layout(title='Price vs points (hover for names)')
fig.show()
```

Check. Hover over the three highest dots in each colour and write down their names.

Day 26: Fixture heatmap

Goal: draw the Fixture Heatmap.

Type.

```
grid = upcoming.pivot_table(index='team_name', columns='event',
                            values='difficulty', aggfunc='mean')
grid = grid.loc[grid.mean(axis=1).sort_values().index]   # easiest teams on
top

fig = px.imshow(grid, text_auto=True, aspect='auto',
                color_continuous_scale='RdYlGn_r',
                labels=dict(x='Gameweek', y='Team', color='Difficulty'))
fig.show()
```

Green means easy and red means hard. A blank cell means no match that week, and a decimal means a double gameweek.

Check. Which team has the best run, and does the chart agree with Day 20?

Day 27: Chart by position

Goal: compare groups.

Type.

```
fig = px.box(value, x='position', y='points_per_million',
hover_name='web_name')
fig.show()
```

Read. The box shows the middle half of players and the dots show unusual ones. Notice that goalkeepers and forwards have different typical values, which is why we should compare players within their own position.

Check. Which position has the widest box?

Day 28: Project Day. Chart functions

Task. Turn your charts into functions that take a table and return a figure:

```
def top_points_chart(df, n=10):
    ...
    return fig

def price_points_chart(df): ...
def fixture_heatmap(upcoming): ...
```

Make each function return a Plotly `fig` rather than calling `fig.show()`. Test all three in the notebook. Streamlit will reuse them next week.

Done when: three functions each produce a chart from the tables, and you can describe each chart's message in one sentence.

Week 5: Your First Web Dashboard

Day 29: Set up your own computer

Goal: move from the browser to a real development setup. Streamlit runs on your own computer, so we bring this step forward to today.

Type.

1. Install **Python 3.11 or newer** from python.org. On Windows, tick **Add Python to PATH** during installation.
2. Install **VS Code** from code.visualstudio.com and add its **Python** extension.
3. Create a folder `fpl-squad-picker`, open it in VS Code, then open the terminal (**Terminal, then New Terminal**).

4. Create a *virtual environment* (a private box for this project's tools) and install the tools:

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Mac or Linux:
source .venv/bin/activate

pip install streamlit pandas requests plotly
```

5. Create `hello.py` containing `print('Hello from my computer')` and run `python hello.py`.

Check. You see the greeting printed. If `python` is not recognised, try `py` or `python3`.

Day 30: Hello Streamlit

Goal: see your first web page made from Python.

Type. Create `app.py`:

```
import streamlit as st

st.set_page_config(page_title='FPL Smart Squad Picker', layout='wide')
st.title('FPL Smart Squad Picker')
st.write('Built with Python and the FPL API.')
```

Run it with `streamlit run app.py`. A browser tab opens. Edit the title, save, and the page updates.

Check. Add `st.header('Player Explorer')` and see it appear.

Day 31: Show the data

Goal: load FPL data in the app.

Type. Create `data.py` and move your loader functions into it. Add Streamlit caching so you do not download the data on every click:

```
import requests
import pandas as pd
import streamlit as st
```

```
BASE = 'https://fantasy.premierleague.com/api/'

@st.cache_data(ttl=3600)
def load_bootstrap():
    return requests.get(BASE + 'bootstrap-static/', timeout=10).json()

@st.cache_data(ttl=3600)
def load_players():
    data = load_bootstrap()
    players = pd.DataFrame(data['elements'])
    teams = pd.DataFrame(data['teams'])[['id', 'name']]
    teams = teams.rename(columns={'id': 'team', 'name': 'team_name'})
    df = players.merge(teams, on='team')
    df['price'] = df['now_cost'] / 10
    df['position'] = df['element_type'].map({1: 'GK', 2: 'DEF', 3: 'MID', 4:
'FWD'})
    df['form'] = pd.to_numeric(df['form'])
    df['points_per_million'] = (df['total_points'] / df['price']).round(2)
    return df
```

In `app.py`, add `from data import load_players` and then `df = load_players()` and `st.dataframe(df[['web_name', 'team_name', 'position', 'price', 'total_points', 'form']])`.

Check. The table appears and you can sort it by clicking a column.

Day 32: Filters

Goal: make the Player Explorer interactive.

Type. Add to `app.py`:

```
st.sidebar.header('Filters')
positions = st.sidebar.multiselect('Position', ['GK', 'DEF', 'MID', 'FWD'],
default=['MID', 'FWD'])
max_price = st.sidebar.slider('Max price', 4.0, 15.0, 15.0, 0.1)
teams_selected = st.sidebar.multiselect('Team',
sorted(df['team_name'].unique()))

view = df[df['position'].isin(positions) & (df['price'] <= max_price)]
if teams_selected:
    view = view[view['team_name'].isin(teams_selected)]

st.write(len(view), 'players match')
st.dataframe(view[['web_name', 'team_name', 'position', 'price',
'total_points', 'form']]
             .sort_values('total_points', ascending=False),
use_container_width=True)
```

Check. Set position to DEF and max price to 5.5. Do the results match what you found on Day 13?

Day 33: Charts inside the app

Goal: show your chart functions in the page.

Type. Put your chart functions from Day 28 into `charts.py`, then in `app.py`:

```
from charts import price_points_chart

st.plotly_chart(price_points_chart(view[view['minutes'] > 450]),
use_container_width=True)
```

Check. Move the price slider and watch the chart change.

Day 34: Tabs and the fixture page

Goal: organize the app.

Type. Add `load_fixtures()` to `data.py` (use your Day 19 code, wrapped in a function that returns `fixtures`). Then restructure `app.py`:

```
tab1, tab2, tab3 = st.tabs(['Player Explorer', 'Value Finder', 'Fixture
Heatmap'])

with tab1:
    ...   # filters, table, scatter chart
with tab2:
    ...   # value table with a minimum-minutes slider
with tab3:
    ...   # heatmap, with a slider for how many gameweeks to show
```

Check. All three tabs open without errors.

Day 35: Project Day. Dashboard version 1

Task.

1. Finish the Value Finder tab with a **minimum minutes** slider and a position selector.
2. Add a short explanation under each tab in plain English, for example *what points per million means*.
3. Take a screenshot of each tab.
4. Write a file `notes.md` listing three things that broke this week and how you fixed them.

Done when: `streamlit run app.py` shows three working tabs on your own computer.

Milestone. You now have a real data app. Everything from here on adds intelligence to it.

Part 4: Machine Learning

Week 6: From Data to Predictions

From here until Day 56 you work in **Google Colab** again, because experiments are easier in notebooks. In Week 9 you move the finished code into your app.

Day 36: What is machine learning?

Goal: understand the idea before the code.

Read. Normally you write rules: *if a player is a striker with good form, expect more points*. In *machine learning* you show the computer thousands of past examples and let it find the rules itself.

Word	Meaning	In our project
Features	The clues we give the model	Recent form, minutes, price, home or away
Target	The thing we want to predict	Points in the *next* gameweek
Training data	Examples the model learns from	Last season's gameweeks
Test data	Examples hidden until the end	This season's gameweeks
MAE	Mean absolute error: the average miss, in points	Lower is better

The golden rule: **always test on data the model has never seen, and always from the future relative to training.**

Type. Open a new notebook `fpl-ml` and download two past seasons from the public archive `vaastav/Fantasy-Premier-League`:

```
import pandas as pd
import numpy as np

RAW = 'https://raw.githubusercontent.com/vaastav/Fantasy-Premier-
League/master/data/'

def load_season(season):
    gw = pd.read_csv(RAW + season + '/gws/merged_gw.csv')
    fx = pd.read_csv(RAW + season + '/fixtures.csv')
    gw['season'] = season
    return gw, fx

gw1, fx1 = load_season('2022-23')
gw2, fx2 = load_season('2023-24')
print(gw1.shape, gw2.shape)
gw1.head()
```

If you get a parsing error, add `on_bad_lines='skip'` inside `read_csv`. If a column name below is missing, print `gw1.columns` and adapt.

Check. Each row is one player in one gameweek. Write down how many rows each season has.

Day 37: Fixture difficulty and the target

Goal: attach difficulty to each row and define what we predict.

Type.

```
def prepare(gw, fx):
    fx = fx[['id', 'team_h_difficulty', 'team_a_difficulty']].rename(columns=
{'id': 'fixture'})
    gw = gw.merge(fx, on='fixture', how='left')
    gw['difficulty'] = gw['team_h_difficulty'].where(gw['was_home'],
gw['team_a_difficulty'])
    gw['price'] = gw['value'] / 10
    gw['home_flag'] = gw['was_home'].astype(int)
    keep = ['season', 'element', 'name', 'position', 'team', 'round',
'minutes',
            'total_points', 'bps', 'price', 'home_flag', 'difficulty']
    return gw[keep]

all_gw = pd.concat([prepare(gw1, fx1), prepare(gw2, fx2)], ignore_index=True)
all_gw = all_gw.sort_values(['season', 'element',
'round']).reset_index(drop=True)

g = all_gw.groupby(['season', 'element'])
all_gw['next_points'] = g['total_points'].shift(-1)       # the target
all_gw['next_home'] = g['home_flag'].shift(-1)
all_gw['next_difficulty'] = g['difficulty'].shift(-1)
all_gw[['name', 'round', 'total_points', 'next_points']].head(8)
```

`shift(-1)` moves values up one row, so each row now also knows the *following* gameweek.

Check. Look at one player's rows. Does `next_points` on row 3 equal `total_points` on row 4?

Simplification to note: double and blank gameweeks make the next row slightly different from the next calendar week. We accept this for now and mention it in the final report.

Day 38: Baselines

Goal: find the score to beat.

Read. A *baseline* is the simplest possible guess. A complicated model that does not beat the baseline is not useful, and this check will teach you more than any algorithm.

Type.

```
data = all_gw.dropna(subset=['next_points']).copy()

def mae(y, pred):
    return float(np.mean(np.abs(y - pred)))

train = data[data['season'] == '2022-23']
test = data[data['season'] == '2023-24']
y = test['next_points']

print('Always predict average:', mae(y, train['next_points'].mean()))
print('Predict last GW points :', mae(y, test['total_points']))
```

We train on 2022-23 and test on 2023-24, so the model never sees the future.

Check. Write both MAE numbers in `experiments.md` (a notes file or a text cell). This is your starting line.

Day 39: Form features

Goal: give the model memory of recent performance.

Type.

```
def add_form(df):
    df = df.sort_values(['season', 'element', 'round']).copy()
    g = df.groupby(['season', 'element'])
    df['form_3'] = g['total_points'].transform(lambda s: s.rolling(3,
min_periods=1).mean())
    df['form_5'] = g['total_points'].transform(lambda s: s.rolling(5,
min_periods=1).mean())
    df['minutes_5'] = g['minutes'].transform(lambda s: s.rolling(5,
min_periods=1).mean())
    df['bps_3'] = g['bps'].transform(lambda s: s.rolling(3,
min_periods=1).mean())
    return df

all_gw = add_form(all_gw)
data = all_gw.dropna(subset=['next_points']).copy()
train = data[data['season'] == '2022-23']
test = data[data['season'] == '2023-24']

print('Rolling 3 baseline:', mae(test['next_points'], test['form_3']))
print('Rolling 5 baseline:', mae(test['next_points'], test['form_5']))
```

A *rolling mean* averages the last few gameweeks. At row *t* it uses only gameweeks up to *t*, which is information we truly have when predicting week *t+1*. That is not leakage.

Check. Which baseline is best now? Record it.

Day 40: Position and clean feature list

Goal: finish the feature table.

Type.

```
print(all_gw['position'].unique())
pos_map = {'GK': 1, 'GKP': 1, 'DEF': 2, 'MID': 3, 'FWD': 4}
all_gw['pos_code'] = all_gw['position'].map(pos_map)

FEATURES = ['form_3', 'form_5', 'minutes_5', 'bps_3', 'price',
            'next_home', 'next_difficulty', 'pos_code']

data = all_gw.dropna(subset=['next_points']).copy()
train = data[data['season'] == '2022-23']
test = data[data['season'] == '2023-24']
print(train[FEATURES].isna().mean().round(3))
```

Check. Which features have missing values, and why? (Hint: a player with no next fixture.)

Day 41: Feature groups

Goal: organize features so you can test them honestly later.

Type.

```
GROUPS = {
    'form': ['form_3', 'form_5', 'bps_3'],
    'minutes': ['minutes_5'],
    'fixture': ['next_home', 'next_difficulty'],
    'market': ['price'],
    'position': ['pos_code'],
}
assert sorted(sum(GROUPS.values(), [])) == sorted(FEATURES)
```

The `assert` line stops the program with an error if your groups and your feature list ever disagree. Writing checks like this is a professional habit.

Check. Add a comment above each group saying why you think it could help.

Day 42: Project Day. Feature table

Task.

1. Put all of Days 36-41 into clean sections in the notebook.
2. Save the table: `data.to_csv('features.csv', index=False)` and download it.
3. In `experiments.md`, make this table with your real numbers:

Date	Method	MAE (test)	Note
	Average of all		
	Last GW points		
	Rolling 3		
	Rolling 5		

Done when: the notebook restarts and runs cleanly, and your table has real values.

Week 7: Models, Honest Testing and Understanding Errors

Day 43: Your first model

Goal: train a model and compare it to the baselines.

Read. *Ridge regression* draws the best straight-line relationship between features and the target. It is simple, fast, and a respectable first model.

Type.

```
from sklearn.linear_model import Ridge

med = train[FEATURES].median()
X_train = train[FEATURES].fillna(med)
X_test = test[FEATURES].fillna(med)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, train['next_points'])
pred_ridge = ridge.predict(X_test)
print('Ridge MAE:', mae(test['next_points'], pred_ridge))
```

Check. Is Ridge better than your best baseline? By how much? Record it, even if the answer is *not much*.

Day 44: A stronger model

Goal: try gradient boosting.

Read. *Gradient boosting* builds many small decision trees, each correcting the errors of the previous ones. It handles curves and interactions that a straight line cannot.

Type.

```
from sklearn.ensemble import HistGradientBoostingRegressor

gb = HistGradientBoostingRegressor(max_depth=4, learning_rate=0.05,
                                   max_iter=200, random_state=0)
gb.fit(train[FEATURES], train['next_points'])
pred_gb = gb.predict(test[FEATURES])
print('Boosting MAE:', mae(test['next_points'], pred_gb))
```

This model accepts missing values directly, so no `fillna` is needed.

Check. Add Ridge and boosting to your experiments table.

Day 45: Measuring what matters

Goal: measure ranking quality, because FPL is about *choosing the best players*, not about tiny average errors.

Read. Most players score 0-2 points, so MAE rewards a model that always guesses low. What you need is a model that puts the *right players at the top*. We test that two ways:

- **Spearman correlation** per gameweek: how well does the predicted order match the real order?
- **Top-10 average:** the real next-week points of the 10 players the model ranks highest.

Type.

```
from scipy.stats import spearmanr

cand = test.assign(pred_gb=pred_gb, pred_ridge=pred_ridge)
cand = cand[cand['minutes_5'] >= 45]     # only players who are really
playing

def summarize(col):
    sp = cand.groupby('round').apply(lambda d: spearmanr(d[col],
d['next_points'])[0]).mean()
    top = cand.groupby('round').apply(lambda d: d.nlargest(10, col)
['next_points'].mean()).mean()
    return round(sp, 3), round(top, 2)

for col in ['form_3', 'pred_ridge', 'pred_gb']:
    print(col, summarize(col))
```

Check. Does the boosting model beat plain `form_3` on the top-10 average? Record both numbers.

Day 46: Which features matter?

Goal: look inside the model.

Type.

```
from sklearn.inspection import permutation_importance

res = permutation_importance(gb, test[FEATURES], test['next_points'],
n_repeats=5,
                             random_state=0,
scoring='neg_mean_absolute_error')
imp = pd.Series(res.importances_mean,
index=FEATURES).sort_values(ascending=False)
print(imp.round(4))
imp.plot(kind='barh', title='Feature importance (higher = more useful)')
```

Permutation importance shuffles one feature at a time and measures how much worse the model gets.

Check. Which feature matters most? Does that match your football intuition?

Day 47: Ablation study

Goal: prove which feature groups help, in the style of a research paper.

Read. An *ablation study* removes one group of features at a time, retrains, and reports the change. It is the standard way to show a part of your system matters.

Type.

```
rows = []
for name, cols in GROUPS.items():
    keep = [f for f in FEATURES if f not in cols]
    m = HistGradientBoostingRegressor(max_depth=4, learning_rate=0.05,
max_iter=200, random_state=0)
    m.fit(train[keep], train['next_points'])
    rows.append((name, mae(test['next_points'], m.predict(test[keep]))))

base = mae(test['next_points'], pred_gb)
ablation = pd.DataFrame(rows, columns=['removed group', 'MAE'])
ablation['change vs full'] = (ablation['MAE'] - base).round(4)
ablation
```

Check. Which group hurts most when removed? Write two sentences in your journal. Do not worry if effects are small, because honest small results are valuable.

Day 48: Error analysis and the leakage test

Goal: learn from mistakes and see why time-based testing matters.

Type.

```
# 1. The 20 worst predictions
err = test.assign(pred=pred_gb)
err['error'] = (err['next_points'] - err['pred']).abs()
err.nlargest(20, 'error')[['name', 'round', 'minutes_5', 'pred',
'next_points']]

# 2. Leakage demo: random split vs time split
from sklearn.model_selection import train_test_split
tr, te = train_test_split(data, test_size=0.3, random_state=0)
m = HistGradientBoostingRegressor(max_depth=4, learning_rate=0.05,
max_iter=200, random_state=0)
m.fit(tr[FEATURES], tr['next_points'])
print('Random split MAE:', mae(te['next_points'], m.predict(te[FEATURES])))
print('Time split MAE  :', base)
```

Check. Look for patterns in the worst predictions, such as big hauls from low-minute players or unexpected benching. Those patterns are ideas for new features. For the leakage test, a random split lets the model peek at neighbouring weeks of the *same player*, so its score is not trustworthy. Write the difference you saw.

Day 49: Project Day. Final model

Task.

1. Retrain the boosting model on **both** seasons (all rows).
2. Save it:

```
import joblib
final = HistGradientBoostingRegressor(max_depth=4, learning_rate=0.05,
max_iter=200, random_state=0)
final.fit(data[FEATURES], data['next_points'])
joblib.dump(final, 'model.joblib')
```

3. Download `model.joblib` and copy your `add_form` function and `FEATURES` list into a new file `features.py` in your `fpl-squad-picker` folder.
4. Finish `experiments.md` with all results, the ablation table, and three sentences on what you learned.

Done when: you have `model.joblib`, `features.py`, and an experiments file with real numbers.

Milestone. You have trained and honestly evaluated a machine learning model. Many people never get this far.